LAB SHEET –7

SQL –DDL, DML, SELECT, Functions, Joins, Subqueries, Indexes, Views and Transactions

NAME : Saurabh Kumar jha

CUID : CU24250005

ROLL NO : 53

COURSE : BTECH CSE 'B'

```
CREATE DATABASE college_db;
USE college_db;
CREATE TABLE departments (
 dept_id INT PRIMARY KEY,
 dept_name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE students (
 student_id INT PRIMARY KEY,
 name VARCHAR(50) NOT NULL,
    age INT CHECK (age >= 17),
    city VARCHAR(50),
    dept_id INT,
 FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

CREATE TABLE courses (
 course_id INT PRIMARY KEY,
    title VARCHAR(80) NOT NULL,
    credits INT DEFAULT 3,
    dept_id INT,
```

```
 FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
 marks INT,
 PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

INSERT SAMPLE DATA
INSERT INTO departments VALUES
(1,'CSE'),
(2,'ECE'),
(3,'Mechanical'),
(4,'Civil');

INSERT INTO students VALUES
(10 1,'Aarav ',2 0,'Roo rk ee',1) ,
(10 2,'Diy a',21 ,'Dehradun',1) ,
(103,'Kabir',19,'Haridwar',2),
(104,'Meera',22,'Roorkee',1),
(10 5,'Rohan',20, 'De lhi',3 ),
(10 6,'Sana',21 ,'De hradun',2) ,
(10 7,'Vik ram',23, 'Jaipur',NULL ),
(10 8,'Isha',1 9,'Roor kee',2) ;
```

```
INSERT INTO courses VALUES
(201,'Data Structures',4,1),
(202,'Operating Systems',4,1),
(203,'Digital Electronics',3,2),
(20 4,'Therm ody nam ic s',3 ,3),
(205,'Machine Learning',4,1);

INSERT INTO enrollments VALUES
(101,201,88),
(101,202,76),
(101,205,91),
(102,201,67),
(102,205,72),
(103,203,81),
(104,201,95),
(104,202,89),
(105,204,58),
(106,203,74),
(108,203,66);

Check Students Table
SELECT * FROM students;

Part A: DDL (Data Definition Language)
A1. Create faculty table
CREATE TABLE faculty (
 faculty_id INT PRIMARY KEY,
```

```
 name VARCHAR(50) NOT NULL,
 email VARCHAR(100) UNIQUE,
 salary DECIMAL(10,2),
    dept_id INT,
 FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

A2. Add phone column

```
ALTER TABLE faculty
ADD phone VARCHAR(15);
```

A3. Rename phone to contact_no

MySQL:

```
ALTER TABLE faculty
RENAME COLUMN phone TO contact_no;
```

PostgreSQL:

```
ALTER TABLE faculty
RENAME COLUMN phone TO contact_no;
```

A4. Drop contact_no column

```
ALTER TABLE faculty
DROP COLUMN contact_no;
```

A5. Drop faculty table

```
DROP TABLE faculty;
```

Difference between DROP, TRUNCATE and DELETE

DROP: Removes the complete table including its structure and data.

TRUNCATE: Removes all rows from the table but keeps the table structure.

DELETE: Removes selected rows or all rows from a table using a condition.

Example:

```
DELETE FROM students
WHERE student_id = 109;
```

Part B: DML (Data Manipulation Language)

B1. Insert new student

```
INSERT INTO students
VALUES (109,'Neha',20,'Roorkee',1);
```

B2. Insert student without specifying city

```
INSERT INTO students
(student_id, name, age, dept_id)
VALUES (110,'Rahul',20,1);
```

To check the value:

```
SELECT * FROM students
WHERE student_id = 110;
```

Observation: Since no city value is provided and there is no DEFAULT constraint on city, the city value is stored as NULL.

B3. Update city of student 105

```
UPDATE students
SET city = 'Roorkee'
WHERE student_id = 105;
```

B4. Increase marks by 5 for course 203

UPDATE enrollments

SET marks = marks + 5

WHERE course_id = 203;

B5. Delete student 109

DELETE FROM students

WHERE student_id = 109;

If student 110 was added for testing:

DELETE FROM students

WHERE student_id = 110;

B6. Try inserting a student with age 15

INSERT INTO students

VALUES (111,'Test',15,'Delhi',1);

Expected error:

The INSERT operation fails because the age column has the CHECK constraint:

CHECK (age >= 17)

Therefore, age 15 is not allowed.

Constraint responsible: CHECK constraint.

Part C: Querying with SELECT

C1. Display all columns of all students

SELECT * FROM students;

C2. Display only name and city

SELECT name, city

FROM students;

C3. List students from Roorkee

```
SELECT *
FROM students
WHERE city = 'Roorkee';
```

C4. Students aged between 20 and 22

```
SELECT * FROM students
WHERE age BETWEEN 20 AND 22;
```

C5. Students from Roorkee or Dehradun

```
SELECT * FROM students
WHERE city IN ('Roorkee','Dehradun');
```

C6. Students whose name starts with A

```
SELECT * FROM students
WHERE name LIKE 'A%';
```

C7. Students having no department

```
SELECT * FROM students
WHERE dept_id IS NULL;
```

C8. Display distinct cities

```
SELECT DISTINCT city
FROM students;
```

C9. Sort students by age descending and name ascending

```
SELECT * FROM students
ORDER BY age DESC, name ASC;
```

C10. Display top 3 oldest students

```
SELECT * FROM students
ORDER BY age DESC
LIMIT 3;
```

C11. Display name and age using aliases

```
SELECT
 name AS "Student Name",
   age AS "Age"
FROM students;
```

Part D: Functions and Aggregation

D1. Find total number of students

```
SELECT COUNT(*) AS total_students
FROM students;
```

D2. Average, minimum and maximum marks

```
SELECT
 AVG(marks) AS average_marks,
 MIN(marks) AS minimum_marks,
 MAX(marks) AS maximum_marks
FROM enrollments;
```

D3. Count students in each city

```
SELECT
   city,
   COUNT(*) AS student_count
FROM students
GROUP BY city;
```

D4. Find average marks for each course

```
SELECT
   course_id,
 AVG(marks) AS average_marks
FROM enrollments
GROUP BY course_id;
```

D5. Cities having more than one student

```
SELECT
    city,
    COUNT(*) AS student_count
FROM students
GROUP BY city
HAVING COUNT(*) > 1;
```

D6. Total credits offered by each department

```
SELECT
    dept_id,
 SUM(credits) AS total_credits
FROM courses
GROUP BY dept_id;
```

D7. Student names in uppercase and length

```
SELECT
 UPPER(name) AS uppercase_name,
 LENGTH(name) AS name_length
FROM students;
```

D8. WHERE vs HAVING

WHERE is used to filter individual rows before grouping and aggregation.

HAVING is used to filter groups after aggregate functions such as COUNT(), AVG(), and SUM() have been applied.

Example:

```
SELECT city, COUNT(*)
FROM students
WHERE age >= 20
GROUP BY city
HAVING COUNT(*) > 1;
```

Here, WHERE filters students based on age, while HAVING filters cities based on the number of students.

Part E: Joins

E1. Student name with department name using INNER JOIN

```
SELECT
 s.name,
 d.dept_name
FROM students s
INNER JOIN departments d
ON s.dept_id = d.dept_id;
```

E2. Display all students including those without department

```
SELECT
 s.name,
 d.dept_name
FROM students s
LEFT JOIN departments d
ON s.dept_id = d.dept_id;
```

Student showing NULL department: Vikram

This is because Vikram has dept_id = NULL.

E3. Display all departments including those with no students

```
SELECT
 d.dept_name,
 s.name
FROM departments d
LEFT JOIN students s
ON d.dept_id = s.dept_id;
```

Observation: Civil department is displayed even though it has no students.

E4. Display student name, course title and marks

```
SELECT
 s.name,
   c.title,
 e.marks
FROM students s
INNER JOIN enrollments e
ON s.student_id = e.student_id
INNER JOIN courses c
ON e.course_id = c.course_id;
```

E5. Students enrolled in Data Structures

```
SELECT s.name
FROM students s
INNER JOIN enrollments e
ON s.student_id = e.student_id
INNER JOIN courses c
ON e.course_id = c.course_id
WHERE c.title = 'Data Structures';
```

E6. Students not enrolled in any course

```
SELECT s.name
FROM students s
LEFT JOIN enrollments e
ON s.student_id = e.student_id
WHERE e.course_id IS NULL;
```

Result: Vikram

E7. Number of students enrolled in each course

```
SELECT
   c.title,
   COUNT(e.student_id) AS student_count
FROM courses c
LEFT JOIN enrollments e
ON c.course_id = e.course_id
GROUP BY c.course_id, c.title;
```

This also includes courses having zero enrollments.

E8. Department with number of students and average age

```
SELECT
 d.dept_name,
   COUNT(s.student_id) AS student_count,
   AVG(s.age) AS average_age
FROM departments d
LEFT JOIN students s
ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name;
```

Part F: Subqueries

F1. Students whose age is greater than average age

```
SELECT *
FROM students
WHERE age > (
   SELECT AVG(age)
 FROM students
```

```
);
F2. Student(s) with highest marks
SELECT
 s.name,
 e.marks
FROM students s
JOIN enrollments e
ON s.student_id = e.student_id
WHERE e.marks = (
 SELECT MAX(marks)
 FROM enrollments
);
F3. Courses having at least one enrollment
SELECT *
FROM courses
WHERE course_id IN (
   SELECT course_id
 FROM enrollments
);
F4. Students enrolled in more than 2 courses
SELECT
 s.name
FROM students s
WHERE s.student_id IN (
   SELECT student_id
 FROM enrollments
   GROUP BY student_id
   HAVING COUNT(course_id) > 2
```

```
);
```

F5. Department with highest number of students

```
SELECT
 d.dept_name,
    COUNT(s.student_id) AS student_count
FROM departments d
LEFT JOIN students s
ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY student_count DESC
LIMIT 1;
```

F6. Students scoring above their course average

```
SELECT
 s.name,
    c.title,
 e.marks
FROM students s
JOIN enrollments e
ON s.student_id = e.student_id
JOIN courses c
ON e.course_id = c.course_id
WHERE e.marks > (
 SELECT AVG(e2.marks)
 FROM enrollments e2
 WHERE e2.course_id = e.course_id
);
```

Explanation: The subquery calculates the average marks for the same course, and the outer query selects students whose marks are greater than that average.

Part G: Indexes, Views and Transactions

G1. Create index on students(city)

```
CREATE INDEX idx_students_city
ON students(city);
```

Run EXPLAIN:

```
EXP LAIN
SELECT *
FROM students
WHERE city = 'Roorkee';
```

Observation: The index gives the database an additional way to find rows based on the city column. For a very small table, the database may still choose a sequential/table scan because it can be faster. The benefit of an index is usually more noticeable with larger tables.

G2. Create student_marks view

```
CREATE VIEW student_marks AS
SELECT
 s.name AS student_name,
   c.title AS course_title,
 e.marks
FROM students s
JOIN enrollments e
ON s.student_id = e.student_id
JOIN courses c
ON e.course_id = c.course_id;
```

G3. Query students scoring above 80

```
SELECT *
FROM student_marks
```

WHERE marks > 80;

G4. Drop view and index

DROP VIEW student_marks;

DROP INDEX idx_students_city;

G5. Transaction using ROLLBACK

START TRANSACTION;

UPDATE enrollments

SET marks = 100

WHERE student_id = 101

AND course_id = 201;

ROLLBACK;

Check the value:

SELECT *

FROM enrollments

WHERE student_id = 101

AND course_id = 201;

Result: The marks remain unchanged because ROLLBACK cancels the update.

Original marks were 88.

G6. Transaction using COMMIT

START TRANSACTION;

UPDATE enrollments

SET marks = 100

WHERE student_id = 101

AND course_id = 201;

COMMIT;

Check the value:

SELECT *

```
FROM enrollments
WHERE student_id = 101
AND course_id = 201;
```

Result: The marks become 100 because COMMIT permanently saves the transaction.

Part H: Challenge Problems

H1. Rank students by total marks

```
SELECT
 s.name,
 SUM(e.marks) AS total_marks,
 RANK() OVER (ORDER BY SUM(e.marks) DESC) AS student_rank
FROM students s
JOIN enrollments e
ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
ORDER BY student_rank;
```

H2. Assign grades using CASE

```
SELECT
   student_id,
   course_id,
 marks,
   CASE
      WHEN marks >= 90 THEN 'A'
      WHEN marks >= 75 THEN 'B'
      WHEN marks >= 60 THEN 'C'
      ELSE 'D'
   END AS grade
```

```
FROM enrollments;
```

H3. Find pairs of students from the same city

```
SELECT
 s1.name AS student1,
 s2.name AS student2,
   s1.city
FROM students s1
JOIN students s2
ON s1.city = s2.city
AND s1.student_id < s2.student_id;
```

H4. Find second highest marks without LIMIT

```
SELECT MAX(marks) AS second_highest_marks
FROM enrollments
WHERE marks < (
 SELECT MAX(marks)
 FROM enrollments
);
```